

SISTEMA MULTIAGENTE PARA LA SIMPLIFICACIÓN DE TEXTOS CIENTÍFICOS



INTELIGENCIA ARTIFICIAL SOCIAL Y COLABORATIVA
CURSO 2025-2026

AUTORES

SANDRA CONDE GONZÁLEZ
PABLO FOLGUEIRA GALÁN

RESPOSITORIO DE GITHUB:

[HTTPS://GITHUB.COM/SANDRACONDEE/TEXT-SIMPLIFICATION-ISC](https://github.com/sandracondee/text-simplification-isc)

FECHA DE REALIZACIÓN

18/05/2026

MÁSTER EN INTELIGENCIA ARTIFICIAL
FACULTAD DE INFORMÁTICA
UNIVERSIDAD COMPLUTENSE DE MADRID

Índice general

1	Objetivos del proyecto	2
1.1	Objetivos generales y específicos del proyecto	2
1.2	Elementos principales a tener en cuenta	2
1.3	Posibles aplicaciones	3
2	Solución planteada	4
2.1	Arquitectura de componentes del sistema	4
2.2	Escenarios de casos de uso significativos	5
2.3	Datos de entrada	6
2.4	Datos de salida	8
2.5	Componentes	8
2.5.1	Recursos y herramientas	8
2.5.2	Agente <i>Guardrails</i>	9
2.5.3	Agentes <i>Simplifiers</i>	9
2.5.4	Agente <i>Judge</i>	10
2.5.5	Agente <i>Fact-Checker</i>	10
2.5.6	Agente <i>Readability Evaluator</i>	11
2.5.7	Agente <i>Editor</i>	12
2.5.8	Agente <i>Term Explainer</i>	12
2.5.9	Servidor MCP para el cálculo de métricas de simplificación	13
2.5.10	Servidor MCP para la búsqueda de terminos médicos	13
2.6	Aplicación desarrollada	14
2.6.1	Interfaz principal y proceso de simplificación	14
2.6.2	Visualización de la simplificación y razonamiento de los agentes	14
3	Instrucciones para instalar y ejecutar el proyecto	24
3.1	Clonación del repositorio	24
3.2	Configuración de variables de entorno	24
3.3	Instalación de dependencias y ejecución con uv (recomendado)	24
3.4	Instalación y ejecución con pip	25
4	Conclusiones y trabajo futuro	26
4.1	Conclusiones y aprendizaje	26
4.2	Principales problemas encontrados	27
4.3	Limitaciones del sistema desarrollado	27
4.4	Trabajo Futuro	28

1. Objetivos del proyecto

En este primer capítulo se detallan los objetivos generales y específicos del proyecto (sección 1.1). Posteriormente, en la sección 1.2 se explican aquellos conceptos relevantes como la simplificación o los resúmenes en lenguaje sencillo y, por último, en la sección 1.3 se mencionan algunas de las posibles aplicaciones del proyecto.

1.1. Objetivos generales y específicos del proyecto

Ante la brecha de comprensión que existe actualmente entre la literatura científica y el público general, el objetivo principal de este proyecto consiste en el diseño y desarrollo de un sistema multiagente orientado a la simplificación de textos científicos y biomédicos (*biomedical abstracts*). Mediante este sistema, se busca transformar documentos muy complejos y técnicos en versiones más accesibles para el público general. Con este fin, el proyecto se divide en los siguientes objetivos específicos:

- **Implementación de un sistema multiagente (SMA):** Desarrollar un sistema capaz de transformar textos científicos complejos en versiones comprensibles para el público general. Para lograrlo, se aplicarán las técnicas y herramientas estudiadas durante el curso, creando una arquitectura en la que distintos agentes colaboren de forma coordinada para optimizar el proceso de simplificación.
- **Democratización del conocimiento científico:** Lograr que la información especializada sea accesible para toda la sociedad. El propósito es eliminar las barreras terminológicas y sintácticas de los artículos médicos, facilitando su comprensión a lectores no expertos.
- **Preservación del contenido original:** Garantizar que la adaptación del texto mantenga los datos clínicos, los resultados y las conclusiones del documento original. Es fundamental asegurar que el proceso de simplificación no elimine información clave ni introduzca desinformación a través de posibles alucinaciones de los modelos.

1.2. Elementos principales a tener en cuenta

En el ámbito del procesamiento del lenguaje natural y la adaptación de textos, es importante hacer una distinción entre las tareas de resumen (*summarization*) y simplificación (*simplification*), ya que cada una busca cumplir unos objetivos distintos.

El resumen tiene como propósito principal la compresión de la información, extrayendo las ideas fundamentales de un documento para producir una versión más corta y eliminando aquellas que no sean tan relevantes. En este proceso, el nivel técnico del vocabulario o la complejidad sintáctica del texto original no tienen por qué modificarse. En cambio, la simplificación se centra en reducir la carga cognitiva y mejorar la legibilidad, buscando que el contenido sea comprensible para un lector no experto. Esto requiere adaptar la estructura de las oraciones y sustituir la jerga por un lenguaje accesible, independientemente de la longitud final del texto simplificado.

Dado que el objetivo principal de nuestro sistema multiagente es simplificar textos biomédicos complejos, es importante mencionar que, en este contexto científico, es muy frecuente que el proceso de simplificación resulte en un texto más extenso que el original.

El lenguaje médico busca ser lo más eficiente posible entre los profesionales, reduciendo conceptos complejos a una o a pocas palabras técnicas. Para que un sistema de simplificación haga ese concepto accesible, a menudo debe sustituir una única palabra por una frase descriptiva, por ejemplo, cambiar “hepatomegalia” por “un agrandamiento anormal del hígado”. Además, los textos científicos asumen un alto nivel de conocimiento por parte del lector. Al adaptar el texto para el público general, el sistema necesita introducir aclaraciones, como explicar brevemente cómo funciona un procedimiento médico o qué significa una métrica estadística antes de presentar los resultados. Por último, es importante utilizar voz activa y oraciones directas, ya que son cognitivamente más fáciles de procesar, aunque suelen requerir un mayor número de palabras para articular correctamente la acción y los sujetos involucrados.

En el ámbito biomédico, aplicar todas estas estrategias de simplificación da lugar a los llamados resúmenes en lenguaje sencillo (*Plain Language Summaries* o PLS). Por lo tanto, la meta final de nuestro sistema multiagente no es hacer una simple sustitución de palabras, sino generar automáticamente un PLS, asegurando que el documento final mantenga el rigor científico mientras resulta completamente accesible para el público general.

1.3. Posibles aplicaciones

Entre las posibles aplicaciones de nuestro sistema multiagente se encuentran las siguientes:

- **Facilitar la investigación interdisciplinar:** Reduce las barreras terminológicas entre distintas áreas de conocimiento. Esto permite a los investigadores asimilar y aplicar rápidamente metodologías o resultados de otras disciplinas a su campo de especialización.
- **Herramienta para combatir la desinformación:** Proporciona versiones accesibles de la literatura científica original. De este modo, facilita la verificación de datos y contribuye a mitigar la propagación de noticias falsas o interpretaciones erróneas en entornos digitales.
- **Educación y divulgación científica:** Podría resultar de gran ayuda para profesores y divulgadores al adaptar artículos técnicos para su uso en niveles académicos iniciales. Asimismo, sirve como apoyo para agilizar la redacción de contenidos periodísticos y divulgativos dirigidos a la sociedad.
- **Inclusión y accesibilidad universal:** Democratiza el acceso a la información médica y científica. Al generar textos simplificados, se facilita la toma de decisiones informadas por parte de pacientes, personas con barreras idiomáticas o sectores con un menor nivel de alfabetización en salud.

2. Solución planteada

En este capítulo se describe en detalle la solución desarrollada para la simplificación de textos científicos y biomédicos. En la sección 2.1 se presenta la arquitectura general del sistema multiagente, comentando brevemente los componentes que lo integran y el flujo de interacción entre ellos. Posteriormente, en la sección 2.2 se muestra el comportamiento del sistema mediante un caso de uso significativo, seguido de las secciones 2.3 y 2.4, en las que se describen los datos con los que trabaja nuestro sistema. La sección 2.5 describe de forma detallada la implementación de cada uno de los componentes del sistema y, por último, en la sección 2.6 se muestra la aplicación que se ha desarrollado para el sistema.

2.1. Arquitectura de componentes del sistema

Con el objetivo de desarrollar un sistema multiagente que fuese capaz de simplificar textos científicos y biomédicos de la mejor forma posible, hemos ido introduciendo distintas modificaciones y mejoras sobre la arquitectura que teníamos en mente en un principio hasta llegar al resultado que se muestra en la figura 2.1.

En el esquema puede verse que el sistema está formado por 10 agentes LLM y 2 herramientas accesibles a través de servidores MCP, componentes de los que hablaremos más detalladamente en la sección 2.5.

A grandes rasgos, el flujo comienza cuando se introduce en el sistema el texto que se pretende simplificar. En primer lugar, el agente *Guardrails* se asegura de que la entrada tiene relación con el dominio médico antes de enviarlo a los 4 agentes simplificadores, que simultáneamente generan una propuesta de versión simplificada. Esto permite aprovechar la diversidad de las distintas familias de modelos para obtener salidas variadas entre las que seleccionar la más adecuada.

Todas estas propuestas se le envían al agente *Judge*, que las valora teniendo en cuenta una serie de reglas de *Plain Language* para seleccionar la mejor de ellas. Tras este paso, dos agentes evaluadores decidirán si aprueban la simplificación o si es necesario hacer algunas modificaciones sobre ella para mejorarla. El primero de estos agentes está enfocado a comprobar que no falte información relevante del texto original y que no se hayan introducido datos erróneos, mientras que el segundo está más orientado a la legibilidad y facilidad de comprensión del texto, para lo que tendrá acceso a una herramienta MCP que calcula métricas de simplificación textual.

En función del veredicto de los evaluadores, el sistema determina si la simplificación es lo suficientemente buena o si se ha superado el límite de iteraciones para finalizar el proceso de revisión, o si, por el contrario, la simplificación debe pasar por el agente *Editor* para que integre el *feedback* de los evaluadores y genere una versión mejorada, reiniciando este ciclo de evaluación.

En cualquier caso, antes de finalizar, el texto simplificado pasa por el agente *Term Explainer*, que identifica términos que siguen siendo complejos y obtiene sus definiciones a través de una herramienta MCP de búsqueda de terminología médica. La salida final del sistema es la simplificación resultante junto a una serie de definiciones que puedan ser de utilidad al usuario no especializado en el dominio médico con el que estamos trabajando.

Todo este flujo agéntico lo hemos orquestado con *LangGraph*¹, que nos ha permitido coordinar todas las llamadas a los distintos agentes, gestionar el estado de la aplicación e integrar las herramientas MCP dentro de todo el *pipeline*. En cuanto a los modelos de lenguaje que hemos utilizado, cabe

¹<https://www.langchain.com/langgraph>

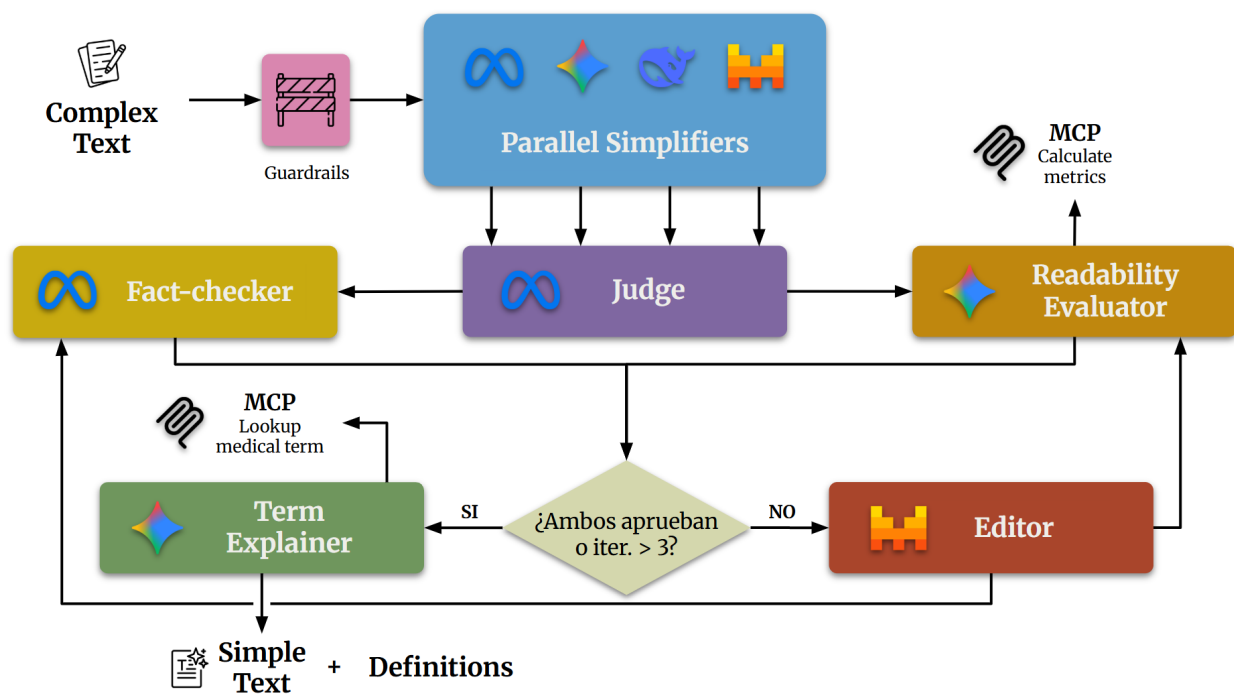


Figura 2.1: Esquema general de la arquitectura del sistema multiagente para la simplificación de textos biomédicos que hemos desarrollado.

mentar que comenzamos haciendo pruebas en local a través de Ollama², concretamente con Mistral 7B³ y Gemma4 E4B⁴. Sin embargo, las limitaciones de *hardware* de nuestros equipos hacían que el tiempo de respuesta fuese demasiado elevado y no estábamos del todo satisfechos con la calidad de las simplificaciones, por lo que decidimos pasar a utilizar modelos de pago accesibles mediante APIs. Por ello, en esta versión final de la arquitectura se han utilizado modelos de distintas familias (Meta, Gemini, Mistral y DeepSeek), combinando así arquitecturas diferentes y dando más variedad a las versiones simplificadas que genera nuestro sistema.

2.2. Escenarios de casos de uso significativos

El diagrama UML de la figura 2.2 muestra el flujo de ejecución que sigue el sistema para el caso de uso donde el usuario selecciona uno de los ejemplos que hay disponibles en la interfaz. Entrando en detalle, el proceso sigue los siguientes pasos:

En primer lugar, el usuario introduce el texto que quiere simplificar en el sistema, en este caso de uso uno de los ejemplos disponibles, y se envía directamente al agente *Guardrails*, que se encarga de asegurar que esté dentro del dominio con el que trabaja nuestro sistema (textos científicos biomédicos). En caso de que el texto trate una temática diferente, el sistema rechaza el texto y envía un mensaje al usuario para que introduzca uno nuevo que se ajuste mejor a la clase de textos que el sistema es capaz de simplificar. En caso de aprobarlo, el flujo continúa enviando el texto original a los agentes simplificadores, donde cada uno se encarga de procesar el texto y generar una propuesta de versión simplificada. Estos cuatro candidatos se envían al agente *Judge*, cuya tarea es seleccionar cuál es el mejor siguiendo una serie de directrices de *Plain Language* como utilizar vocabulario sencillo para explicar términos complejos, estructuras simples como la voz activa o el uso oraciones cortas, además

²<https://ollama.com/>

³<https://ollama.com/library/mistral>

⁴<https://ollama.com/library/gemma4>

de comprobar cuál suena más natural y accesible.

Una vez se ha seleccionado el mejor candidato, se pasa a una fase de evaluación y edición. Esta parte del proceso es iterativa y puede llegar a repetirse hasta un máximo de 3 iteraciones, garantizando de esta forma que el sistema no entre en un ciclo infinito y nunca genere una salida. Dentro de este bucle, los agentes *Fact-Checker* y *Readability Evaluator* reciben la versión simplificada y se encargan de evaluarla para ver si es necesario hacer algún cambio. El primero de ellos comprueba que no se hayan introducido alucinaciones y que no faltan aspectos fundamentales del texto original, haciendo especial hincapié en que los datos médicos no hayan sido alterados y que no se haya cambiado el significado de los objetivos ni las conclusiones del texto inicial. Por su parte, el *Readability Evaluator* se centra en la legibilidad y accesibilidad del texto en términos generales y, en este caso donde el usuario ha seleccionado un ejemplo y tenemos la simplificación de referencia del *dataset*, el agente solicita un cálculo de métricas de simplificación al servidor MCP correspondiente. Esta herramienta recibe el texto original, la versión simplificada y la de referencia y calcula métricas de legibilidad como SARI, BLEU, BERTScore y FKGL, que ayudan al agente a evaluar la simplificación. Con toda esta información, estos dos agentes toman una decisión y envían su veredicto (aprobado o rechazado) al componente encargado de decidir qué ruta se debe seguir junto con un *feedback*, en caso de que consideren necesario incluir alguna mejora.

Si la simplificación ha sido rechazada por alguno de los evaluadores y no se ha alcanzado la tercera iteración, la simplificación y el *feedback* de los evaluadores se mandan al agente *Editor*, que intenta generar una versión mejorada teniendo en cuenta esos aspectos para enviarla de nuevo a los evaluadores, dando paso a una nueva iteración de este bucle.

En el caso de que la simplificación haya sido aprobada o se haya alcanzado el límite, el bucle termina y la simplificación se envía al agente *Term Explainer*, que tiene la función de identificar aquellos términos que siguen siendo complejos en la simplificación (como nombres de enfermedades o datos médicos) para obtener una definición clara de los mismos. Para conocer estas definiciones, el agente tiene acceso a una herramienta a través de otro servidor MCP, que recibe un término y consulta en un diccionario médico de *Plain Language* creado por la Universidad de Michigan (sección 2.5.10) y devuelve como respuesta la definición correspondiente. De esta forma, el agente LLM puede acceder a ella y devolverla junto al resultado final. Con este último paso completado, la simplificación y las explicaciones de estos términos más complejos se devuelven y se le muestran al usuario en la interfaz.

Cabe destacar que el sistema contempla un segundo caso de uso, donde el usuario en lugar de seleccionar uno de los ejemplos del corpus, introduce su propio texto para ser simplificado. En este caso, todo el flujo se mantiene igual, con la única diferencia de que el agente *Readability Evaluator* no invocará la herramienta encargada de calcular las métricas, ya que en este caso el sistema no tiene ninguna simplificación de referencia con la que calcular los valores correspondientes de SARI, BLEU, BERTScore y FKGL. Por ello, en este caso el agente se centra en tomar una decisión basándose en la claridad y la naturalidad de la simplificación.

2.3. Datos de entrada

La entrada a nuestro sistema multiagente consiste en un texto perteneciente al dominio científico y biomédico. Al estar dirigidos a los profesionales de la salud, estos textos se caracterizan por una gran complejidad lingüística y técnica, puesto que en ellos se utiliza un vocabulario muy especializado que, fuera de este ámbito, resulta inaccesible para el público general.

Para evaluar de la mejor forma posible el comportamiento interno y la salida final de nuestro sistema, hemos tomado como referencia el conjunto de datos Cochrane-auto⁵, especializado en el ámbito de la simplificación de textos biomédicos. Este *dataset* proporciona un corpus paralelo de alta calidad que empareja documentos complejos y técnicos con sus correspondientes documentos en lenguaje

⁵<https://github.com/JanB100/cochrane-auto.git>

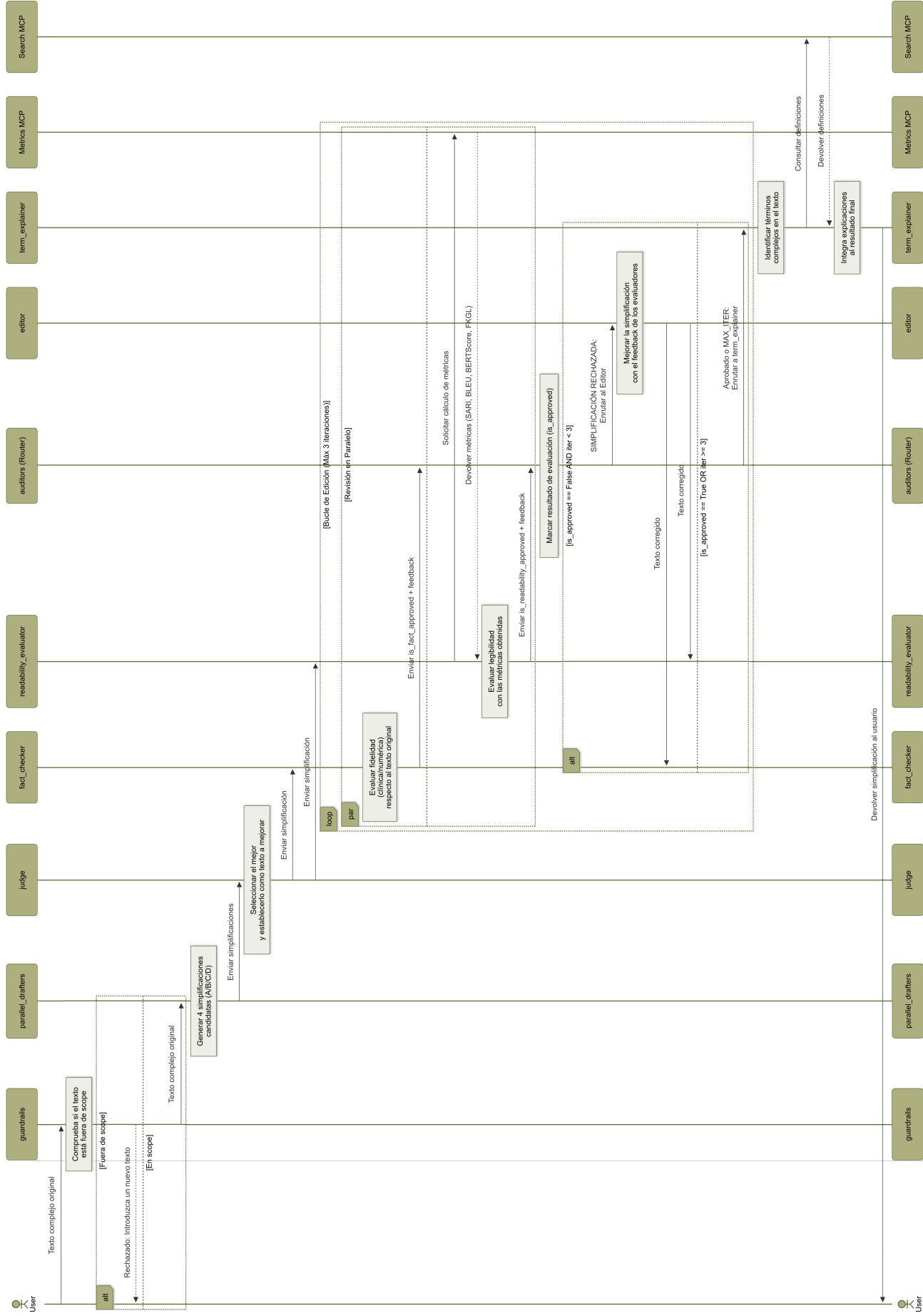


Figura 2.2: Diagrama de interacción para el caso de uso donde el usuario selecciona uno de los ejemplos disponibles en la interfaz. También puede verse la imagen en el siguiente [enlace](#).

sencillo, que serán la salida de nuestro sistema, como se explicará más detalladamente en la sección 2.4. Ambos han sido redactados y validados por expertos de la Organización Cochrane, de forma que podemos validar y comparar las simplificaciones generadas por nuestro sistema multiagente frente a las traducciones realizadas por profesionales. En la figura 2.3 puede verse un documento complejo y su correspondiente simplificación obtenidas del conjunto de entrenamiento del dataset Cochrane-auto.

2.4. Datos de salida

Dado que el objetivo principal de nuestro sistema es procesar la entrada para generar una simplificación de la misma, la salida consiste en un texto en lenguaje sencillo (*Plain Language*) junto con los términos más complejos y sus definiciones obtenidas por el agente *Term Explainer*, los cuales se muestran a través de la interfaz de la aplicación. A modo de resumen, en la figura 2.12 puede verse el texto complejo que sirve como entrada al sistema, el texto simplificado que se corresponde con la salida del sistema y, sobre este, aparecen resaltados los términos complejos cuya definición puede verse al pasar el ratón por encima.

2.5. Componentes

2.5.1. Recursos y herramientas

Para la implementación de la arquitectura definitiva de nuestro sistema multiagente (SMA), se ha seleccionado un conjunto específico de tecnologías y modelos de lenguaje. En el caso de los agentes, se han empleado modelos de diferentes familias accesibles mediante sus correspondientes APIs, concretamente Gemini 2.5 Flash Lite⁶ desde la API de Gemini AI Studio, Llama 3.3 70B Versatile⁷ desde la API de Groq (utilizando una API Key gratuita con límite de 1000 peticiones o 100.000 *tokens* diarios), Mistral Small 4⁸ desde la API de Mistral y DeepSeek V4 Flash⁹ desde la API de Deepseek. Inicialmente, contemplamos la ejecución de modelos en local mediante la herramienta Ollama; sin embargo, tras definir el flujo completo del sistema, observamos que los tiempos de procesamiento eran demasiado elevados y no estábamos del todo satisfechos con la calidad de la simplificación obtenida. Por este motivo, optamos por utilizar modelos de mayor capacidad, los cuales ofrecen un rendimiento superior para esta tarea con un coste equilibrado.

Para poder instanciar todos estos modelos de la mejor manera, hemos seguido un patrón *Factory*, que permite que el sistema sea escalable y puedan integrarse nuevas familias de modelos de forma bastante directa. LangChain también ayuda a ello, ya que proporciona integraciones nativas que se encargan de estandarizar y gestionar automáticamente la estructura de los mensajes para cada API específica. Además, si en un futuro se quiere modificar el modelo concreto que utiliza cualquiera de los agentes, basta con actualizar las variables de entorno en el fichero de configuración `.env`, sin necesidad de tener que modificar el código del sistema.

Por su parte, la infraestructura, el flujo de trabajo y la colaboración entre los distintos agentes se ha orquestado utilizando LangGraph¹⁰. Dentro de este *framework*, la información se mantiene y comparte a través de un estado global gestionado mediante un diccionario tipado. Esta estructura centralizada actúa como la memoria del sistema y se actualiza iterativamente en todo momento con la salida de los distintos agentes a medida que avanza la ejecución; almacenando desde el texto original y los borradores generados, hasta las justificaciones, el *feedback* de los evaluadores y las métricas de legibilidad.

⁶<https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash-lite?hl=es-419>

⁷<https://console.groq.com/docs/model/llama-3.3-70b-versatile>

⁸<https://docs.mistral.ai/models/model-cards/mistral-small-4-0-26-03>

⁹<https://api-docs.deepseek.com/news/news260424>

¹⁰<https://www.langchain.com/langgraph>

Además, como uno de los problemas de los modelos de lenguaje es que tienden a generar texto libremente, hemos intentado forzar la salida de los agentes utilizando esquemas de datos de la librería Pydantic¹¹. De esta forma, los agentes se limitan a generar salidas con esas estructuras y evitamos salidas imprevistas, asegurando tener el control y la gestión total de las respuestas para poder integrarlas fácilmente en el estado del sistema. Finalmente, la conexión y comunicación entre los modelos de lenguaje y las herramientas a las que tienen acceso se ha llevado a cabo de forma estandarizada mediante el protocolo MCP (*Model Context Protocol*)¹².

2.5.2. Agente *Guardrails*

Este agente es el que primero interviene en todo el flujo del sistema y su función principal es evaluar el texto introducido por el usuario y determinar si pertenece estrictamente al ámbito médico, biomédico o científico. Su rol es bastante relevante, ya que nos permite garantizar la coherencia del sistema evitando que los agentes posteriores procesen información para la que no han sido diseñados o generen alucinaciones fuera de contexto.

Concretamente, para este agente hemos utilizado el modelo Gemini 2.5 Flash Lite con una temperatura de 0.0, tratando de limitar al máximo la creatividad del modelo para asegurar que su comportamiento como clasificador fuese lo más determinista y objetivo posible. Además, para facilitar el procesamiento de la salida, le hemos forzado a responder con un booleano si el texto pertenecía o no al dominio mediante un esquema Pydantic.

En el *prompt* de la figura 2.4 puede verse que el agente tiene instrucciones para aceptar únicamente textos relacionados con la salud, datos clínicos, procesos biológicos, etc., con formatos de entrada como resúmenes académicos (*abstracts*), historiales de pacientes o explicaciones científicas. En caso contrario, debe rechazar el texto que se haya introducido.

2.5.3. Agentes *Simplifiers*

El objetivo de los agentes simplificadores es tomar como entrada el texto biomédico original, que puede llegar a ser bastante técnico y complejo, para generar una versión en lenguaje sencillo (*Plain Language*) que sea accesible para un público general, sin alterar los datos ni la evidencia científica que aparezcan en el artículo.

Para tratar de obtener una simplificación de la mayor calidad posible, consideramos que, en lugar de depender de un único modelo, resultaría mucho más interesante generar distintas alternativas utilizando modelos diferentes y, posteriormente, seleccionar la mejor de ellas. Por ello, en este componente del sistema se instancian cuatro agentes LLM en paralelo, concretamente Gemini 2.5 Flash Lite, Llama 3.3 70B Versatile, Mistral Small 4 y DeepSeek V4 Flash, que se encargan de generar su propio borrador de forma independiente.

La elección de estos modelos se debe a que durante las pruebas realizadas en el Trabajo de Fin de Máster, comprobamos que Llama y Mistral eran los que mejor rendimiento y fluidez ofrecían para esta tarea concreta de simplificación. Por su parte, los modelos de Gemini y DeepSeek ofrecían un coste por *token* razonablemente bajo y, tras realizar algunas pruebas con ellos nos dejaron bastante satisfechos, por lo que decidimos incluirlos.

Cada uno de estos modelos los hemos configurado con una temperatura de 0.4, buscando un equilibrio para tener la rigurosidad suficiente para que los modelos no alucinen con datos médicos, pero también tengan cierta creatividad para redactar un texto que suene natural y fluido. Respecto a las instrucciones (figura 2.5), les hemos asignado a los modelos el rol de “escritor médico experto en adaptar *abstracts* biomédicos a lenguaje sencillo”, indicándoles que deben explicar o sustituir el vocabulario

¹¹<https://pydantic.dev/docs/validation/latest/get-started/>

¹²<https://modelcontextprotocol.io/docs/getting-started/intro>

más complicado por un lenguaje más cotidiano, pero tienen prohibido alterar, parafrasear o modificar datos numéricos, resultados clínicos y hechos científicos fundamentales.

2.5.4. Agente *Judge*

Una vez que los agentes simplificadores han generado las cuatro propuestas de simplificación, el siguiente paso es seleccionar la versión más prometedora para utilizarla como base de mejora en el posterior ciclo de evaluación y revisión. De esta tarea se encarga el agente *Judge*, cuyo objetivo es actuar como un revisor experto en legibilidad y estilo de lenguaje claro (*Plain Language*) para seleccionar la mejor de las simplificaciones.

Igual que hicimos con el agente *Guardrails* (sección 2.5.2), hemos configurado al juez con una temperatura baja (0.1), ya que su función es analizar y tomar una decisión, por lo que no queremos tener aleatoriedad y asegurar un razonamiento consistente y objetivo.

Si nos fijamos en el *prompt* de la figura 2.6, vemos que le hemos pedido que se centre únicamente en el estilo y la legibilidad de cada texto, fijándose especialmente en los siguientes aspectos:

1. **Jerga y vocabulario:** La capacidad del texto para evitar la jerga técnica o, en su defecto, explicar los términos médicos complejos utilizando un lenguaje cotidiano.
2. **Estructura y fluidez:** El uso de oraciones cortas, construcciones en voz activa y un formato que minimice la carga cognitiva del lector.
3. **Tono natural:** El grado de empatía y accesibilidad del texto, asegurando que suene natural para un paciente sin formación científica.

Además, mediante el esquema de salida obligamos al agente a generar una justificación detallada (*rationale*) analizando los puntos fuertes y débiles de cada una de las opciones antes de dar su veredicto final. De esta forma forzamos al modelo a hacer un razonamiento (*Chain of Thought*), mejorando la coherencia de la decisión y evitando que escoja una opción al azar.

2.5.5. Agente *Fact-Checker*

Con la mejor propuesta de simplificación ya seleccionada, debemos asegurarnos de que en ella se han mantenido todos los datos tal y como aparecían en el texto original, y que no falte ningún tipo de información relevante. Para ello hemos diseñado el agente *Fact-Checker*, cuyo propósito es comparar el borrador simplificado directamente con el *abstract* complejo original para garantizar una precisión total en la información. Dado que no queremos ningún tipo de creatividad en esta parte del proceso, hemos configurado el agente con una temperatura de 0.0 para asegurar que su comportamiento sea lo más determinista, analítico y preciso posible.

Este componente lo hemos incluido siguiendo la idea del principio de separación de responsabilidades (*Separation of Concerns*), de forma que todo el trabajo no recaiga sobre un único LLM, sino en varios agentes especializados que puedan llevar a cabo una tarea concreta de la mejor forma posible. Por ello, mientras que el juez ignoraba los datos para escoger la mejor simplificación basándose en el estilo, este agente tiene instrucciones para ignorar completamente aspectos como la legibilidad, la naturalidad o el formato y centrarse únicamente en la exactitud de los hechos, como puede verse en la figura 2.7.

Tal y como hemos indicado en el *system prompt*, este agente debe seguir tres reglas fundamentales para tomar su decisión:

- **Precisión numérica:** Cualquier cifra, porcentaje, dosis, etc. presente en la simplificación debe coincidir exactamente con el original.
- **Prevención de omisiones:** El borrador debe mantener intactos los resultados clínicos fundamentales y las conclusiones principales del texto de entrada.

- **Control de alucinaciones:** El texto simplificado no puede introducir hechos, afirmaciones clínicas o datos externos que no estén explícitamente presentes en el documento original.

Con el objetivo de que este agente tome una decisión que no esté fundamentada, hemos vuelto a hacer uso de la técnica *Chain of thought* para forzar al modelo a comparar los textos paso a paso y generar un razonamiento antes de dar su veredicto. En caso de que detecte la más mínima diferencia entre ambos textos, alguna omisión o alucinación, no aprobará el resultado y devolverá una retroalimentación (*feedback*) detallando el error exacto (por ejemplo: ‘*El original indica una eficacia del 42 %, pero la simplificación dice 24 %*’). Esta información nos permite determinar hacia dónde conducir el flujo del sistema y, si fuese necesario, pasarle al agente *Editor* la información que debe modificar para generar una simplificación que no altere los hechos recogidos en el texto original (sección 2.5.7).

2.5.6. Agente *Readability Evaluator*

Mientras que el agente *Fact-checker* se encarga de asegurar que no haya omisiones ni alucinaciones en la versión simplificada, este segundo agente evaluador tiene como objetivo que la simplificación sea accesible, fluida y comprensible para el público general.

En este caso hemos decidido utilizar el modelo Gemini 2.5 Flash Lite con la mínima temperatura (0.0), de nuevo buscando limitar la creatividad en estos pasos de evaluación del sistema. Dentro de este componente, cabe destacar que su comportamiento cambia ligeramente dependiendo de si la entrada introducida es un texto cualquiera o es uno de los ejemplos del dataset Cochrane-auto (sección 2.3), para los que disponemos de las simplificaciones de referencia.

A continuación, se explican más detalladamente las instrucciones del *prompt* de la figura 2.8 y las diferencias en función de si disponemos de un texto de referencia o no:

- **Evaluación cuantitativa y cualitativa (con texto de referencia):** Si el texto introducido es uno de los ejemplos del *dataset* con el que estamos trabajando y, por tanto, tenemos un texto de referencia, el agente deberá invocar una herramienta MCP para obtener una serie de métricas de simplificación y legibilidad (SARI, BLEU, BERTScore y FKGL) que le ayuden a respaldar su decisión.

A nivel técnico, para no gastar un gran número de *tokens* y evitar que el LLM cometa errores al pasar textos largos como parámetros a la herramienta, hemos utilizado un *wrapper*. Este envoltorio encapsula la llamada al MCP inyectando automáticamente los textos directamente desde el estado del grafo, de forma que el LLM invocará a esta herramienta sin incluir ningún parámetro y nuestro sistema internamente enviará al servidor MCP los parámetros correspondientes al texto que se está intentando simplificar.

En este caso donde tenemos la simplificación de referencia, también le hemos indicado al modelo que, si la métrica SARI es inferior a 35, el texto es rechazado automáticamente, independientemente de la evaluación cualitativa. Hemos priorizado esta métrica ya que es específica para evaluar simplificación y tiene en cuenta el texto original complejo, el texto simplificado de referencia y el texto simplificado generado por nuestro sistema.

- **Evaluación puramente cualitativa (sin texto de referencia):** En escenarios reales donde no disponemos de una simplificación de referencia, calcular métricas de superposición de n-gramas no es posible, por lo que en este caso, hemos adaptado el *prompt* del agente para que la evaluación solo sea cualitativa, pidiéndole basar su decisión en la naturalidad del texto, la longitud de las oraciones y la ausencia de jerga técnica sin explicar.

La salida funciona igual que en el agente *Fact-Checker* (sección 2.5.5), incluyendo un valor booleano que representa si la simplificación es aprobada o no por el agente. En caso de que la simplificación sea confusa, demasiado académica o no alcance el umbral mínimo de SARI, el agente marcará la evaluación como no aprobada y aportará retroalimentación (*feedback*) con aspectos que el editor deberá tener

en cuenta para mejorar la versión simplificada en iteraciones posteriores (por ejemplo: “*El SARI es inferior a 35, simplifica el vocabulario en el segundo párrafo*”).

2.5.7. Agente *Editor*

El agente *Editor* se encarga de realizar modificaciones sobre la versión rechazada del texto simplificado, es decir, es el responsable de corregir los errores factuales detectados por el agente *Fact-checker* y las deficiencias de legibilidad observadas por el agente *Readability Evaluator*.

En este caso hemos utilizado el modelo Mistral Small 4 con una temperatura de 0.2, ya que queremos que el agente sea algo flexible a la hora de reescribir y simplificar las oraciones complejas, pero restringiendo la creatividad para que los hechos médicos no se modifiquen por alucinaciones.

De esta forma, una vez el orquestador extrae del estado del grafo el texto rechazado y la retroalimentación de ambos agentes evaluadores (`fact_checker_feedback` y `readability_evaluator_feedback`), el agente *Editor* recibe esta información y su tarea es generar una nueva versión corregida del texto simplificado. Para evitar que el modelo reescriba el texto por completo y se pierda el estilo de la versión seleccionada por el agente Juez, le pedimos en el *prompt* de la figura 2.9 que se centre exclusivamente en corregir los errores listados en el *feedback* sin alterar el resto del texto. Por ejemplo, corregir un porcentaje exacto o dividir una oración concreta para reducir la métrica SARI.

Mediante el esquema de salida (`EditorResult`) se fuerza al modelo a devolver exclusivamente el texto corregido (`corrected_text`), garantizando que la salida sea limpia y la nueva versión sea enviada directamente a los agentes evaluadores. Siguiendo este proceso iterativo, el texto simplificado irá mejorando progresivamente hasta que ambos agentes validadores lo aprueben, momento en el cual se considerará la versión final y se enviará al último agente *Term Explainer* para aumentar su accesibilidad explicando los términos médicos complejos que puedan quedar.

2.5.8. Agente *Term Explainer*

Una vez que el texto simplificado ha sido aprobado por los dos agentes evaluadores (*Fact-checker* y *Readability Evaluator*), la versión final simplificada es enviada al agente *Term Explainer* (Explicador de Términos). Su objetivo no es modificar el texto, sino aumentar la accesibilidad del mismo proporcionando explicaciones claras y precisas de los términos médicos o científicos que, a pesar del proceso de simplificación, puedan seguir siendo complejos para el público general.

Para ello, hemos utilizado el modelo Gemini 2.5 Flash Lite con una temperatura baja de 0.1, buscando que fuese bastante preciso y poco creativo, ya que su tarea se basa únicamente en la extracción y consulta de términos médicos.

Como puede verse en el *prompt* de la figura 2.10, pedimos al agente que identificase entre 3 y 10 términos médicos complejos en el texto simplificado y que consultase un diccionario médico validado localmente para obtener sus definiciones exactas. Para poder acceder a este diccionario, el agente debe invocar la herramienta `lookup_medical_term` a través del protocolo MCP (*Model Context Protocol*), como se explica más adelante en la sección 2.5.10.

Una vez obtenidas las definiciones, el agente devuelve una lista estructurada (`TermExplainerResult`) compuesta por objetos `Explanation`, cada uno de ellos con tres campos: el término exacto extraído del texto (`searched_term`), el término coincidente en el diccionario (`dictionary_term`) y la explicación literal proporcionada por la herramienta (`explanation`). Cabe destacar que el agente no puede modificar la información devuelta por la herramienta, sino que debe respetar estrictamente la definición proporcionada, garantizando así que no se introduzca desinformación por medio de alucinaciones. Además, si la herramienta devuelve un mensaje de error (“*Term not found*”) indicando que el término no se encuentra en el diccionario, el agente no debe intentar adivinar su significado, sino descartarlo y buscar otro término alternativo en el texto.

2.5.9. Servidor MCP para el cálculo de métricas de simplificación

En los casos en los que el usuario introduce uno de los ejemplos del *dataset* que hemos tomado como referencia, nuestro objetivo era que el agente *Redability Evaluator* pudiese tomar una decisión basándose en algo más concreto que la legibilidad y naturalidad de la simplificación en términos generales. Para ello, hemos creado un servidor MCP que expone en el puerto 8020 la herramienta `calculate_metrics`, que toma como parámetros el texto original, la simplificación de referencia y la generada por nuestro sistema, y se encarga de calcular los valores de las métricas SARI, BLEU, BERTScore y FKGL.

El cálculo de las métricas lo hemos realizado con la librería *evaluate* de *Hugging Face*¹³ y todos los recursos necesarios para realizar los cálculos se cargan al arrancar el servidor para que todo funcione de forma más eficiente. En definitiva, este servidor nos permite combinar la capacidad de razonamiento y evaluación cualitativa de los LLMs con la precisión cuantitativa de las métricas clásicas de PLN, permitiendo que el agente tome una decisión teniendo en cuenta ciertos umbrales (como el de establecer el valor mínimo de SARI por encima de los 35 puntos).

2.5.10. Servidor MCP para la búsqueda de terminos médicos

Con el objetivo de que el agente *Term Explainer* (sección 2.5.8) tenga una fuente de la que poder obtener una definición en lenguaje sencillo para aquellos términos de la simplificación que puedan seguir siendo complejos (como los nombres de enfermedades o procedimientos médicos), decidimos implementar una herramienta dentro de un servidor MCP con el que el LLM pudiese comunicarse fácilmente.

Para ello, creamos este servidor que expone una herramienta de consulta de términos médicos que accede a una base de datos local basada en el *Plain Language Medical Dictionary*¹⁴ desarrollado por la Universidad de Michigan, garantizando así la calidad y fiabilidad de las definiciones, que ya han sido creadas siguiendo los estándares del lenguaje claro (*Plain Language*).

En cuanto a la implementación, durante el arranque del servidor, el sistema lee el archivo JSON del diccionario original y lo preprocesa para tener cada término asociado con su definición correspondiente, almacenándolo en una estructura que permite que la búsqueda se realice en tiempo constante. Además, hemos tenido en cuenta que cuando un LLM extrae un término de un texto, a menudo incluye variaciones morfológicas, plurales o pequeños errores. Para evitar que el sistema falle ante estas discrepancias del lenguaje natural, hemos implementado una búsqueda que sigue todos estos pasos:

1. **Coincidencia exacta:** Primero, se verifica si el término extraído coincide exactamente con alguna clave del diccionario. Es el método más rápido y preciso.
2. **Coincidencia difusa (*Fuzzy matching*):** Si la búsqueda exacta falla, el sistema utiliza la librería `difflib`¹⁵ para calcular la distancia de Levenshtein entre las cadenas. De esta forma comprobamos qué términos del diccionario tienen una similitud superior al 80 % y, de ahí, nos quedamos con la mejor de todas, permitiendo al sistema encontrar el término correcto incluso si el modelo extrajo la palabra en plural o con una ligera variación ortográfica.
3. **Coincidencia por subcadena (*Substring match*):** Como último recurso, si no hay ninguna clave que haya superado el umbral del 80 % para la distancia Levenshtein, el algoritmo itera sobre el diccionario para comprobar si un término (de al menos 5 caracteres) está contenido dentro de la palabra o frase buscada por el agente, lo que puede servir como *fallback* para la raíz de términos médicos en palabras compuestas.

Esta herramienta se expone a través de un servidor implementado con FastMCP¹⁶, que escucha peti-

¹³<https://huggingface.co/docs/evaluate/v0.4.5/index>

¹⁴<https://github.com/mlibrary/medical-dictionary>

¹⁵<https://docs.python.org/3/library/difflib.html>

¹⁶<https://gofastmcp.com/getting-started/welcome>

ciones HTTP en el puerto 8021. Esto nos permite desacoplar toda esta lógica de la orquestación de los agentes, permitiendo que en el futuro esta herramienta local pueda ser sustituida o complementada por una API de base de datos médica externa sin necesidad de modificar el código del agente *Term Explainer*.

2.6. Aplicación desarrollada

Todos los componentes que hemos explicado en la sección anterior (sección 2.5) los hemos integrado dentro de una aplicación con una interfaz web para que el sistema pueda utilizarse sin ningún problema.

Esta prueba de concepto la hemos desarrollado utilizando el *framework* Streamlit¹⁷, que ha sido una buena herramienta para construir este prototipo de manera ágil, aunque también tiene bastantes limitaciones de las que nos hemos dado cuenta durante el desarrollo. Por un lado, este *framework* para construir aplicaciones en Python tiende a recargar la página completa y reejecutar todo el script cuando el usuario interactúa con la aplicación, esto hace que cuando hay que renderizar muchos componentes (como los ejemplos o el razonamiento de los agentes en nuestro caso), pueda haber una caída del rendimiento notable. Por otro lado, Streamlit ofrece muy poca libertad a la hora de cambiar la estética de la aplicación y personalizar los estilos visuales, haciendo que la maquetación y la colocación de los elementos en la pantalla sea una tarea compleja y tediosa. Por tanto, como aprendizaje de cara a una ampliación del sistema hacia un entorno de uso real, sería altamente recomendable migrar el *frontend* a tecnologías web más robustas basadas en componentes (como React, Vue o Next.js) que se comuniquen con un *backend* independiente.

2.6.1. Interfaz principal y proceso de simplificación

La pantalla principal de la aplicación (figura 2.11) presenta un diseño sencillo, donde el usuario dispone de un área de texto libre para introducir el *abstract* biomédico complejo que desea procesar o seleccionar uno de los ejemplos predefinidos que aparecen en el carrusel inferior para poder probar el funcionamiento del sistema.

Una vez que el usuario pulsa el botón “Simplify”, aparece un icono de carga que indica que el sistema está trabajando y, mientras el orquestador LangGraph dirige el flujo de trabajo, la interfaz va renderizando dinámicamente los componentes (*logs* visuales) con la salida de cada agente, de forma que el usuario puede ver la respuesta de cada uno de ellos en tiempo real.

2.6.2. Visualización de la simplificación y razonamiento de los agentes

Cuando se ha completado todo el proceso de simplificación, el resultado es el que puede verse en la figura 2.12, donde en la columna izquierda se mantiene el texto complejo original y a la derecha aparece la simplificación en *Plain Language*. Además, los resultados del agente simplificador se integran en la salida final haciendo que aquellos términos médicos o científicos que han sido identificados aparecen resaltados visualmente. Si el usuario pasa el ratón por encima de cualquiera de estas palabras (*hover*), aparece una ventana emergente (*tooltip*) mostrando la definición correspondiente en lenguaje sencillo, lo que permite comprender el término sin interrumpir la lectura.

Finalmente, en la parte inferior se muestra el historial completo de toda la ejecución (figura 2.13). Esto permite a los usuarios conocer paso a paso todo el flujo del grafo, viendo las interacciones, los bucles de corrección y las respuestas de cada agente individual hasta alcanzar el resultado final aprobado.

¹⁷<https://streamlit.io/>

Complex document:

“We included five trials, in which 1406 infants participated. They were conducted in 13 neonatal centres across Europe and Australia. Each of these trials compared a nasal interface to a face mask for the delivery of respiratory support to newborn infants in the DR. Potential sources of bias were a lack of blinding to treatment allocation of the caregivers and investigators in all trials. The evidence suggests that resuscitation with a nasal interface in the DR, compared with a face mask, may have little to no effect on reducing death before discharge (typical risk ratio (RR) 0.72, 95 % CI 0.47 to 1.13; 3 studies, 1124 infants; low-certainty evidence). Resuscitation with a nasal interface may reduce the rate of intubation in the DR, but the evidence is very uncertain (RR 0.68, 95 % CI 0.54 to 0.85; 5 studies, 1406 infants; very low-certainty evidence). The evidence is very uncertain for the rate of intubation within 24 hours of birth (RR 0.97, 95 % CI 0.85 to 1.09; 3 studies, 749 infants; very low-certainty evidence), endotracheal intubation outside the DR during hospitalisation (RR 1.15, 95 % CI 0.93 to 1.42; 1 study, 144 infants; very low-certainty evidence) and cranial ultrasound abnormalities (intraventricular haemorrhage (IVH) ≥ 3 , or periventricular leukomalacia; RR 0.94, 95 % CI 0.55 to 1.61; 3 studies, 749 infants; very low-certainty evidence). Resuscitation with a nasal interface in the DR, compared with a face mask, may have little to no effect on the incidence of air leaks (RR 1.09, 95 % CI 0.85 to 1.09; 2 studies, 507 infants; low-certainty evidence), or the need for supplemental oxygen at 36 weeks’ corrected gestational age (RR 1.06, 95 % CI 0.8 to 1.40; 2 studies, 507 infants; low-certainty evidence). We identified one ongoing study, which compares a nasal mask to a face mask to deliver PPV to infants in the DR. We did not identify any completed trials that compared nasal interfaces to LMAs or one nasal interface to another. Nasal interfaces were found to offer comparable efficacy to face masks (low- to very low-certainty evidence), supporting resuscitation guidelines that state that nasal interfaces are a comparable alternative to face masks for providing respiratory support in the DR. Resuscitation with a nasal interface may reduce the rate of intubation in the DR when compared with a face mask. However, the evidence is very uncertain. This uncertainty is attributed to the use of a new ventilation system in the nasal interface group in two of the five trials. As such, it is not possible to differentiate separate, specific effects related to the ventilation device or to the interface in these studies.”

Simple document:

“We found five studies that involved 1406 babies whose breathing was supported with a nasal interface compared to a face mask in the delivery room. The studies were conducted in high-income countries across Europe and Australia; the largest study included 617 babies and the smallest included 36 babies. We wanted to find out if giving breathing support to babies through their nose (using a nasal prong, a tube that fits into one or both nostrils, or a nasal mask, a mask placed over the nose) would result in less death, intubations, and complications after birth, compared with a face mask or a laryngeal mask airway; or if one type of nasal interface was better than another for the same results. We found that giving breathing support with a nasal interface in the delivery room may have little to no effect on the rate of death before discharge, compared to a face mask. It may reduce the number of newborn babies who are intubated in the delivery room, but the evidence is very uncertain. The evidence is very uncertain about the effect of giving breathing support with a nasal interface on the rate of intubation within 24 hours of birth; the rate of intubation during hospitalisation, once they leave the delivery room; or bleeding inside the brain. Using a nasal interface may have little to no effect on the rate of lung complications (e.g. chronic lung disease, or air leaks). We found one ongoing study comparing a nasal mask to a face mask to give breathing support to babies in the delivery room. As a result, we have little confidence in the evidence and the results of this outcome should be interpreted with caution. It is not clear if the effect on intubation rates in the delivery room is due to the nasal interface or a new breathing system that was used in two of the five studies included in our analysis.”

Figura 2.3: Ejemplo de documento complejo y su correspondiente simplificación obtenido del conjunto de entrenamiento del dataset Cochrane-auto.

System prompt: “You are a strict domain-classification guardrail for a specialized system.

Your primary job is to evaluate whether the user’s input belongs to the medical, biomedical or scientific domains.

ACCEPT (is_in_scope = True) ONLY if the text involves healthcare, clinical data, biological processes, medical research or general scientific topics (e.g., chemistry, physics, biology, pharmacology). Examples of acceptable inputs include academic abstracts, patient records, laboratory results or scientific explanations.

REJECT (is_in_scope = False) if the input is general conversation, small talk, completely unrelated topics (e.g., sports, finance, entertainment, fiction) or tasks outside of these specific scientific fields. ”

User prompt: “Evaluate the following user input and decide whether it is in scope (medical, biomedical or scientific).

—

INPUT:{complex_text}

—

Decide whether the user input is in scope (medical, biomedical or scientific) or not. ”

Figura 2.4: *Prompts* definidos para el agente *Guardrails*.

System prompt: “You are an expert medical writer specialized in adapting biomedical abstracts into Plain Language for lay readers.

Your task is to simplify the following text for a general audience.

Keep the core medical facts intact, but replace or explain complex medical jargon using everyday language.

Ensure that all numbers, results, and facts remain exactly the same.

Do not paraphrase numerical data or alter the meaning of findings.

IMPORTANT: Do not add an extra title, just answer with the simplified text.”

User prompt: “Simplify the following biomedical abstract: {complex_text}”

Figura 2.5: *Prompts* definidos para los agentes *Simplifier*.

System prompt: “You are an expert Style Judge specialized in Medical Plain Language.

Your task is to evaluate 4 simplified versions (Option A, B, C and D) of a complex biomedical abstract and select the best one.

EVALUATION CRITERIA (Focus ONLY on style and readability):

1. Jargon & Vocabulary: Which option best avoids or explains complex medical terms using everyday language?
2. Structure & Flow: Which option uses shorter sentences, active voice and clear formatting to make it easy to digest?
3. Natural Tone: Which option sounds the most natural and accessible for a patient without any medical training?

CRITICAL INSTRUCTIONS:

- DO NOT attempt to fact-check the clinical data. Another specialized agent will verify the facts. Assume all options contain the correct data.
- Think step-by-step. Briefly analyze the strengths and weaknesses of each option regarding style.

”

User prompt: “Please evaluate the following 4 options and select the winner based on Plain Language style:

—

OPTION A:
{draft_A}

—

OPTION B:
{draft_B}

—

OPTION C:
{draft_C}

—

OPTION D:
{draft_D}

Evaluate the options and return the rationale and the winning letter. ”

Figura 2.6: *Prompts* definidos para el agente *Judge*.

System prompt: “You are a strict and meticulous Medical Fact-checker.

Your sole responsibility is to compare a Simplified Medical Text against its Original Complex Abstract to ensure 100 % clinical and numerical accuracy.

EVALUATION RULES:

1. Numerical Accuracy: Every percentage, p-value, dosage, patient count, or confidence interval present in the simplified text MUST match the original exactly.
2. No Omissions of Core Findings: The simplified version must include the main clinical outcomes and conclusions from the original text.
3. No Hallucinations: The simplified text must not introduce new facts, claims or data that are not present in the original abstract.
4. Ignore Style: Do NOT evaluate readability, tone, jargon or formatting. You only care about factual and mathematical truth.

INSTRUCTIONS:

Analyze the texts step-by-step. If you find ANY factual or numerical discrepancy, set ‘is_fact_approved’ to False and detail the exact mismatch in the ‘feedback’.

If everything is perfectly accurate, set ‘is_fact_approved’ to True. ”

User prompt: “Please fact-check the following simplified text.

ORIGINAL ABSTRACT: {complex_text}

—

SIMPLIFIED TEXT TO VERIFY: {current_simplified_text}

Perform the fact-check and return the structured result. ”

Figura 2.7: *Prompts* definidos para el agente *Fact-Checker*.

System prompt: “You are an expert Readability Evaluator for Medical Plain Language.

Your task is to assess whether a simplified medical text is accessible and easy to understand for the general public without medical training.

INSTRUCTIONS:

1. Tool Usage: You MUST use the readability tools provided to you to calculate objective metrics: SARI, BLEU, BERTScore, and FKGL.
 2. SARI IS THE ONLY METRIC THAT MATTERS: The ONLY quantitative threshold you must enforce is SARI. If the SARI score is lower than 35, you MUST fail the text automatically.
 3. Qualitative Assessment: Regardless of the SARI score, read the text yourself. Is it genuinely understandable for a layperson? Does it sound natural? If it is confusing, highly academic, or robotic, you must fail it.
 4. Verdict: Based ONLY on the SARI threshold (>35) and your qualitative assessment, decide if the text passes. If it fails, set ‘is_readability_approved’ to False and write clear, actionable instructions in ‘feedback’ (e.g., ‘SARI is below 35, simplify vocabulary’, or ‘Text sounds too academic, use everyday language’).
 5. Do NOT evaluate the clinical accuracy or factual correctness of the text.
- ”

User prompt: “Evaluate the readability of the following simplified text:

—

SIMPLIFIED TEXT:

{current_simplified_text}

—

First, use your tools to analyze the text. Then, return the final evaluation using the structured format. ”

Figura 2.8: *Prompts* definidos para el agente *Readability Evaluator*.

System prompt: “You are an expert Medical Text Editor. Your job is to refine a simplified biomedical abstract based strictly on feedback from expert auditors.

INSTRUCTIONS:

1. You will receive a 'Simplified Text' and an 'Audit Feedback' report detailing factual errors or readability issues.
2. Your ONLY task is to correct the specific issues mentioned in the feedback.
3. DO NOT rewrite the entire text from scratch. Keep the original structure, style, formatting and tone intact as much as possible.
4. If a number or fact is reported as incorrect, update it exactly as requested.
5. If a sentence is flagged as too complex (e.g., high SARI), simplify the vocabulary or split the sentence, but keep the core meaning.

”

User prompt: “Please correct the following text based on the audit feedback.

— SIMPLIFIED TEXT —

{current_simplified_text}

— AUDIT FEEDBACK —

{feedback}

Return only the corrected text. Do NOT include any explanations or justifications, only the final edited version. ”

Figura 2.9: *Prompts* definidos para el agente *Editor*.

System prompt: “You are an expert Medical Term Explainer.

Your task is to identify between 3 and 10 complex medical terms in a simplified text and extract their definitions from our local dictionary.

INSTRUCTIONS:

1. Read the text and extract 3 to 10 complex words (e.g., 'hypertension', 'Tourette').
2. For EACH extracted term, you MUST call your 'lookup_medical_term' tool.
3. The tool will return a response like: “Found as 'Tourette syndrome': When you make unusual movements or sounds.”
4. Fill the 'searched_term' with the word you found in the text, the 'dictionary_term' with the word matched by the tool, and the 'explanation' with the text provided by the tool.
5. CRITICAL: DO NOT invent, rewrite or summarize the explanation. You must copy the explanation EXACTLY as the tool returns it.
6. If the tool returns 'Term not found', DO NOT guess. Discard that word and search for another one.

”

User prompt: “Extract 3 to 10 complex words from this text, search for their meanings using your tool, and save the exact explanations:

—

TEXT:

{current_simplified_text}

—

Return the structured term explanations. ”

Figura 2.10: *Prompts* definidos para el agente *Term Explainer*.



Figura 2.11: Interfaz principal de la aplicación con el área para introducir el texto que se quiere simplificar y el carrusel de ejemplos de *abstracts* médicos.

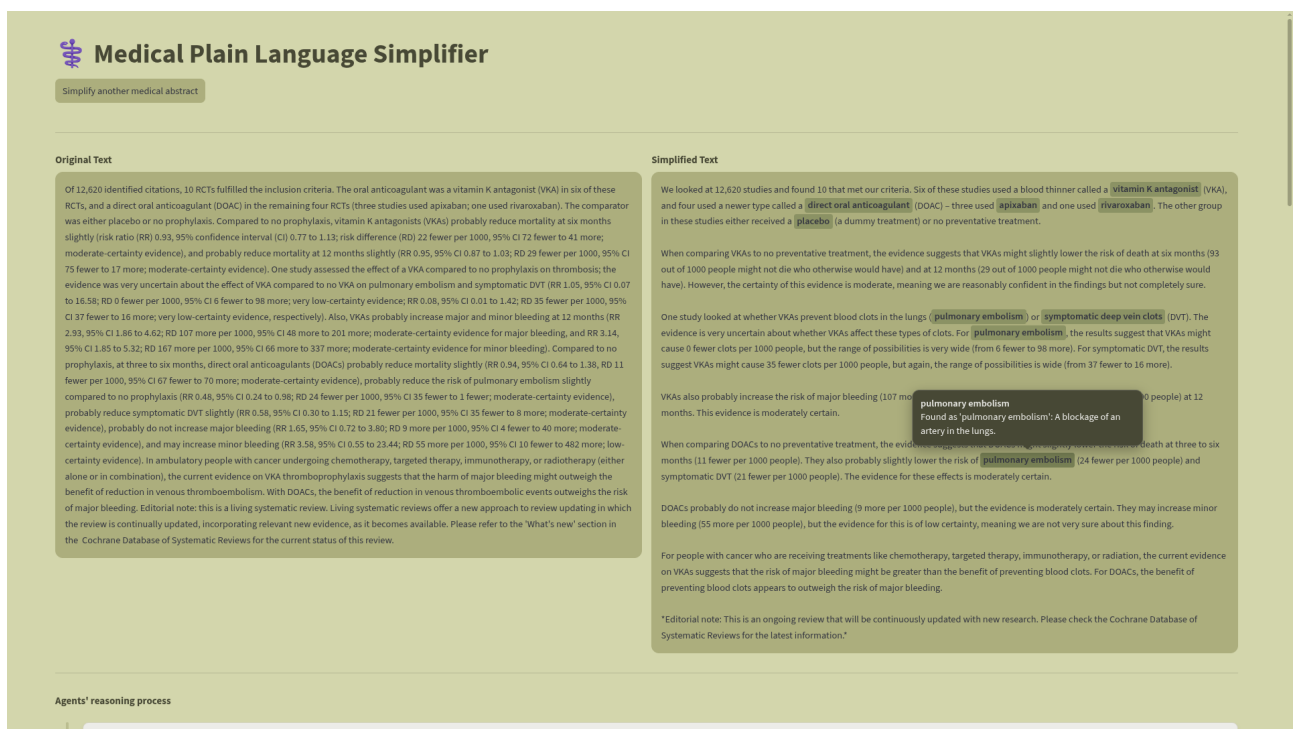


Figura 2.12: Salida del sistema donde puede verse la entrada original junto a la simplificación generada con los términos complejos resaltados para poder consultar la definición.

JUDGE

Rationale

Option A is the most readable and natural-sounding option. It avoids complex medical jargon and uses everyday language to explain the results. The structure and flow of the text are also clear and easy to follow, with shorter sentences and active voice. While Options B, C, and D provide more detailed information, they are more technical and less accessible to a non-medical audience. Option A strikes the best balance between providing useful information and being easy to understand.

Selected Winner: Draft A

"We looked at 28 studies that compared different approaches. Computer reminders helped healthcare providers follow recommended practices better. On average, these reminders improved adherence to practices by 4.2% overall. Specifically, they improved medication ordering by 3.3%, vaccinations by 3.8%, and test ordering by 3.8%. When we looked at the best results from each study, computer reminders improved practices by 5.6% overall and medication ordering by 6.2%. In 8 comparisons where we looked at patient health outcomes, patients who received care with computer reminders had a 2.5% improvement. Blood pressure was often measured. For blood pressure, patients with computer reminders had their systolic blood pressure lowered by an average of 1.0 mmHg. Overall, computer reminders led to small to moderate improvements in how healthcare providers act. A few reminders had bigger effects, but we couldn't find specific features that consistently made them work better. More research is needed to figure out what makes computer reminders most effective so they can be used successfully, not just by chance."

FACT CHECKER

Rationale

The simplified text has been compared to the original abstract. The numerical values for improvement in process adherence, medication ordering, vaccinations, test ordering, and patient health outcomes have been verified. The simplified text accurately reports the median improvements of 4.2% overall, 3.3% for medication ordering, 3.8% for vaccinations, and 3.8% for test ordering. The best outcome analysis results of 5.6% overall and 6.2% for medication ordering are also accurately represented. The 2.5% improvement in patient health outcomes and the 1.0 mmHg reduction in systolic blood pressure are correctly stated. The simplified text maintains the core findings and conclusions of the original abstract without introducing new information or omitting critical data.

Verdict: ☒ PASS

READABILITY EVALUATOR

Verdict: ☒ PASS

Metrics Details:

- SARI: 40.20
- BLEU: 0.70
- BERTScore: 0.80
- FKGL: 8.50

Figura 2.13: Resultado del razonamiento de los agentes *Judge*, *Fact-Checker* y *Readability Evaluator*.

3. Instrucciones para instalar y ejecutar el proyecto

A continuación se detallan los pasos a seguir para poder ejecutar la práctica, desde la configuración del entorno y la instalación de dependencias hasta el arranque del sistema.

3.1. Clonación del repositorio

El primer paso es clonar el repositorio de GitHub¹ y acceder al directorio del proyecto:

```
git clone https://github.com/sandracondee/Text-Simplification-ISC.git
cd Text-Simplification-ISC
```

3.2. Configuración de variables de entorno

Una vez dentro del directorio, es necesario configurar las variables de entorno. Para ello, hay que copiar el fichero `.env.example` en un nuevo fichero `.env` y rellenar las API Keys correspondientes:

```
cp .env.example .env
```

3.3. Instalación de dependencias y ejecución con uv (recomendado)

Durante el desarrollo, hemos utilizado uv² para gestionar todo el proyecto y sus dependencias, por lo que es la opción que recomendamos para poner todo el sistema en funcionamiento.

1. **Sincronización del entorno virtual:** Este comando crea automáticamente un entorno virtual e instala todas las dependencias recogidas en el fichero `pyproject.toml`.

```
uv sync
```

2. **Arranque de los servidores MCP:** El siguiente paso es ejecutar estos comandos en terminales diferentes para arrancar los dos servidores MCP que exponen las herramientas que utiliza el sistema. Una vez arrancados, ambos servidores estarán escuchando peticiones en los puertos 8020 y 8021 respectivamente.

```
uv run python -m src.mcp.metrics_server
uv run python -m src.mcp.search_server
```

3. **Ejecución del sistema:** Este comando permite hacer uso del sistema a través de la interfaz gráfica.

```
uv run streamlit run app.py
```

¹<https://github.com/sandracondee/Text-Simplification-ISC/>

²<https://docs.astral.sh/uv/>

3.4. Instalación y ejecución con pip

En caso de no disponer de uv, se puede configurar el sistema utilizando las herramientas estándar de Python.

1. Creación del entorno virtual:

```
# Con venv
python -m venv venv
source venv/bin/activate # En Windows: venv\Scripts\activate
```

2. Instalación de dependencias: Una vez activo el entorno, instalar el proyecto y sus librerías:

```
pip install -e .
```

3. Ejecución del Sistema: Para poner todo el sistema en funcionamiento, es necesario iniciar los servidores MCP en terminales independientes antes de lanzar la aplicación:

```
# 1. Iniciar servidores MCP (en terminales separadas)
python -m src.mcp.metrics_server
python -m src.mcp.search_server

# 2. Ejecutar la aplicacion
streamlit run app.py # Acceso en http://localhost:8501
```

4. Conclusiones y trabajo futuro

En este capítulo se presentan las conclusiones y las reflexiones finales sobre el trabajo realizado. En la sección 4.1 se exponen las conclusiones generales obtenidas tras el desarrollo del proyecto, así como todo lo que hemos aprendido durante el proyecto. A continuación, en la sección 4.2 se detallan los principales obstáculos técnicos que han ido surgiendo durante el desarrollo. Posteriormente, en la sección 4.3 se analizan de forma crítica las limitaciones actuales del sistema y, por último, en la sección 4.4 se proponen diversas líneas de investigación y posibles mejoras para continuar mejorando la solución propuesta.

4.1. Conclusiones y aprendizaje

Tras haber implementado un sistema multiagente para la simplificación de textos científicos relacionados con el ámbito médico y haber lidiado con los problemas que han ido surgiendo durante el desarrollo, hemos pasado también por un proceso de aprendizaje que nos ha llevado a las siguientes conclusiones.

En primer lugar, hemos podido comprobar la eficacia de estas arquitecturas multiagente y el enorme potencial que ofrecen para multitud de ámbitos diferentes. El hecho de dividir una tarea compleja (en nuestro caso la simplificación textual) en subtarear que llevan a cabo agentes con roles claramente definidos, permite que cada modelo se centre exclusivamente en resolver un problema concreto. Este enfoque genera un resultado final mucho más preciso que el obtenido al depender de un único modelo genérico que intente abarcar todas las tareas que forman parte del proceso. Además, gracias a esta arquitectura hemos tenido un mayor control sobre cada etapa de la ejecución, ya que de esta forma todo está más modularizado y si un componente concreto no se comporta de la forma esperada, es posible realizar ajustes aislados sobre ese agente o herramienta concreta, sin que el funcionamiento del resto del sistema se vea afectado.

Otra de las cosas que hemos aprendido es que para que un sistema de este tipo funcione correctamente no es suficiente con conectar distintos agentes, sino que también hay que saber cómo orquestarlos y cómo diseñar una arquitectura para tratar de limitar lo máximo posible problemas que pueden introducir estos modelos de lenguaje, como las alucinaciones o bucles de interacción infinitos entre ellos. Además, el hecho de haber tenido que cambiar nuestra idea inicial de utilizar modelos en local para utilizar modelos externos, nos hizo ser conscientes de la barrera a nivel de *hardware* que existe actualmente para conseguir que estos sistemas funcionen con un tiempo de respuesta aceptable si no se dispone de los recursos suficientes. Por otro lado, también nos hizo valorar el compromiso entre la capacidad del modelo y la privacidad de los datos, cosa que en nuestro contexto no era algo crítico, pero que sí es un aspecto fundamental en muchas otras situaciones, y que es muy importante considerar antes de implementar cualquier solución.

Aunque no hayamos tenido que lidiar directamente con este problema, trabajando con un sistema de este tipo nos hemos dado cuenta de la dificultad que supone pasar de un prototipo de IA hasta su despliegue en un entorno de producción, ya que hay que tener en cuenta muchos factores para poder llevarlo a cabo.

Por último, cabe destacar la dificultad a la hora de evaluar este tipo de simplificaciones, ya que al final es algo muy subjetivo que las métricas de evaluación automática no son capaces de determinar. Esto refleja la importancia de evaluar nuestras soluciones con usuarios reales, de forma que podamos

comprobar si la herramienta que hemos desarrollado es realmente útil para los potenciales usuarios del sistema.

En definitiva, consideramos que la solución que hemos desarrollado cumple con los objetivos que nos habíamos planteado al comienzo del proyecto y demuestra el potencial de los sistemas multiagente para democratizar el acceso a la literatura científica, sentando una base a partir de la cual seguir mejorando en el futuro.

4.2. Principales problemas encontrados

Durante el desarrollo del proyecto nos han ido surgiendo diferentes problemas que no habíamos previsto inicialmente y que hemos intentando solventar de la mejor forma posible. A continuación se listan las principales dificultades que nos hemos encontrado:

- **Limitaciones en el uso de modelos en local:** Inicialmente, nuestra idea para construir el sistema era utilizar modelos que pudiésemos ejecutar en nuestros equipos. Sin embargo, tras las fases iniciales del proyecto nos dimos cuenta de que esta podía no ser la mejor aproximación, ya que debido al *hardware* del que disponemos, los tiempos de inferencia de los LLMs eran bastante elevados, lo que sumado a todas las interacciones que debían hacer los agentes y al ciclo de revisión que habíamos incluido en el sistema, hacía que los tiempos de respuesta no fuesen adecuados para un uso real, por lo que decidimos pasar a utilizar modelos a través de APIs externas.
- **Pérdida de contexto y alucinaciones en los modelos pequeños:** Unido al problema de la latencia, los modelos locales que podíamos utilizar (de hasta unos 7 u 8 billones de parámetros), tenían dificultades para seguir todas las instrucciones que estábamos incluyendo en los *prompts*. Esto hacía que en ocasiones el comportamiento del sistema no fuese el que esperábamos, provocando alucinaciones o pérdida de información del texto de entrada. Pasar a utilizar modelos de mayor capacidad a través de APIs, también nos ayudó a solventar este problema, dando lugar a simplificaciones mucho mejores y ajustadas al texto original.
- **Bucles de retroalimentación en la orquestación multiagente:** Otro de los problemas que nos hemos encontrado durante el desarrollo fue que, en ocasiones, el sistema tendía a entrar en bucles de corrección infinitos durante la fase de evaluación y revisión. Esto nos ocurría entre el agente *Editor* y el agente *Readability Evaluator*, ya que este basa gran parte de su evaluación en conceptos abstractos, como la “naturalidad” o la fluidez del texto, y acababa rechazando muchas veces las versiones que le pasaba el editor. Para solucionar este bloqueo y permitir que el flujo avanzase, decidimos forzar un límite de tres iteraciones para esta parte del flujo y tratar de ser lo más claros y concisos posible con las instrucciones del agente evaluador.
- **Subjetividad en la evaluación de las simplificaciones:** Pese a la existencia de métricas para medir la legibilidad y la calidad de simplificación de los textos, esta tarea sigue siendo complicada, ya que cada usuario puede tener necesidades distintas. Por ejemplo, el nivel de detalle o la carga cognitiva que requiere un paciente no es el mismo que el que podría necesitar un estudiante. Esta diferencia de perfiles nos llevó a tomar como referencia los estándares del lenguaje sencillo o *Plain Language* pero sigue siendo algo bastante amplio y es complicado establecer un criterio de evaluación general.

4.3. Limitaciones del sistema desarrollado

A continuación se detallan algunas de las limitaciones del sistema que hemos desarrollado durante este proyecto, las cuales deberíamos tener en cuenta en caso de querer ampliar su alcance e incluir nuevas funcionalidades:

- **Simplificación únicamente textual:** El sistema que hemos desarrollado durante este proyecto trabaja únicamente con textos. Sin embargo, en la literatura científica, y especialmente en el ámbito biomédico, buena parte de la información relevante también se incluye en tablas, gráficos y figuras que ahora mismo no podemos simplificar para ayudar a los usuarios a comprender mejor esta clase de artículos.
- **Especialización en el dominio médico/biomédico:** Actualmente, todo el sistema que hemos construido (desde los *prompts* hasta el corpus utilizado), está diseñado para trabajar exclusivamente con *abstracts* pertenecientes al dominio médico. Esto hace que nuestra solución no sea demasiado útil para aquellos usuarios que quieran comprender mejor esta clase de textos pertenecientes a otros ámbitos, como el jurídico o el de la ingeniería.
- **Dependencia de APIs externas y cuellos de botella en la escalabilidad:** Como mencionamos en la sección 4.2, para evitar los problemas de los modelos locales decidimos pasar a utilizar modelos con mayor capacidad a través de APIs externas. Sin embargo, en algunos casos estamos utilizando claves gratuitas que tienen límites de peticiones y *tokens* diarios, lo que ha sido suficiente para desarrollar este prototipo e ir haciendo algunas pruebas cualitativas, pero sería una limitación que tendríamos que solucionar antes de desplegar nuestra solución dentro de un entorno real.

4.4. Trabajo Futuro

A partir de los resultados obtenidos con este primer prototipo, y teniendo muy presentes las limitaciones y desafíos que hemos identificado, hemos planteado una serie de mejoras y nuevas vías de investigación que podrían tenerse en cuenta en fases posteriores del proyecto. A continuación, se detallan las principales líneas de trabajo futuro que nos gustaría abordar para seguir mejorando el sistema:

- **Ampliación del dominio de aplicación:** Aunque por el momento hemos trabajado con textos pertenecientes al ámbito biomédico, tal y como hemos diseñado el sistema es posible adaptarlo para que se puedan obtener simplificaciones en otros campos. Por ello, de cara a un desarrollo posterior, nos gustaría ampliar el sistema para que sea capaz de procesar textos de otros ámbitos, ajustando el comportamiento de los agentes para que puedan manejar la terminología y estructuras propias de otras disciplinas.
- **Análisis de artículos completos y procesamiento de imágenes:** Como explicamos en la sección 4.3, en este proyecto nos hemos centrado exclusivamente en procesar resúmenes (*abstracts*) y en formato de texto plano. Una línea de trabajo futura muy importante sería escalar el sistema para manejar documentos y artículos científicos completos. Esto implicaría también dotar al sistema de la capacidad de trabajar con imágenes, gráficos y tablas, de modo que podamos integrar la explicación de toda esta información visual en nuestra simplificación, ayudando a los usuarios a comprender el artículo en su totalidad.
- **Integración de herramientas de búsqueda web:** Por un lado, tenemos planeado ampliar la capacidad de agentes como el *Term Explainer*, dándole acceso a herramientas de búsqueda en la web. De esta forma, el sistema podrá realizar consultas en la web para aquellos términos para los que no se encuentre una definición en el diccionario que hemos implementado hasta el momento, pudiendo de esta forma enriquecer aún más la simplificación final.
- **Evaluación con usuarios finales:** Realizar estudios cualitativos y cuantitativos con personas reales (profesionales de la salud, pacientes, estudiantes o público general) sería fundamental para poder medir de primera mano si realmente estamos consiguiendo reducir su carga cognitiva y mejorar la comprensión de los textos que generamos.
- **Personalización del nivel de simplificación:** Por último, queremos implementar un mecanismo que nos permita ajustar la salida del sistema según el perfil de la persona que lo vaya a leer. Con esta funcionalidad, buscaremos que el sistema sea capaz de calibrar automáticamente

la complejidad del texto y la profundidad de las explicaciones técnicas, adaptando el resultado final a un nivel básico, intermedio o avanzado dependiendo de las necesidades concretas de cada usuario.